

From Notebook to Production

Two End-to-End Machine Learning Case Studies

House Price Prediction • Customer Churn Prediction

AI/ML Engineering Curriculum — Projects 1 & 2

ml-systems repository

Table of Contents

TOC \h \o "1-2"

Introduction

This ebook documents two projects from a sixteen-project AI/ML engineering curriculum, each built end to end: raw data in, a served prediction out, with every stage in between — exploratory analysis, feature engineering, model comparison, hyperparameter tuning, explainability, and deployment — implemented, tested, and verified against real data rather than left as notebook sketches.

The two projects were chosen to sit at opposite ends of classical supervised learning. Project 1 is a regression problem on a small, clean, extensively studied dataset (Ames Housing). Project 2 is a binary classification problem on a moderately imbalanced dataset (Telco Customer Churn) where the interesting engineering work happens after the model trains — in choosing a decision threshold and translating it into a dollar figure a non-technical stakeholder can act on.

Both projects follow the same architectural shape, deliberately: data loading with a no-credentials fallback, EDA paired with formal hypothesis tests (not just plots), a leakage-free feature pipeline persisted as a single reusable artifact, a multi-model comparison with cross-validation, automated hyperparameter search, model explainability via SHAP, and a served interface (a REST API for one, an interactive dashboard for the other). Seeing the same shape solve two different problem types is the point — it's evidence the shape generalizes, not that either project was fitted to look good in isolation.

Each part below closes with a case study: a narrative extension of the technical results into a hypothetical but grounded business scenario, using the project's actual numbers as evidence rather than invented statistics. Every dollar figure in a case study is explicitly labeled as illustrative where it is — the goal is to show how the technical results connect to a business decision, not to claim more precision than the underlying assumptions support.

Part I — House Price Prediction

A Full ML Pipeline for Regression on the Ames Housing Dataset

1. Problem Statement & Business Context

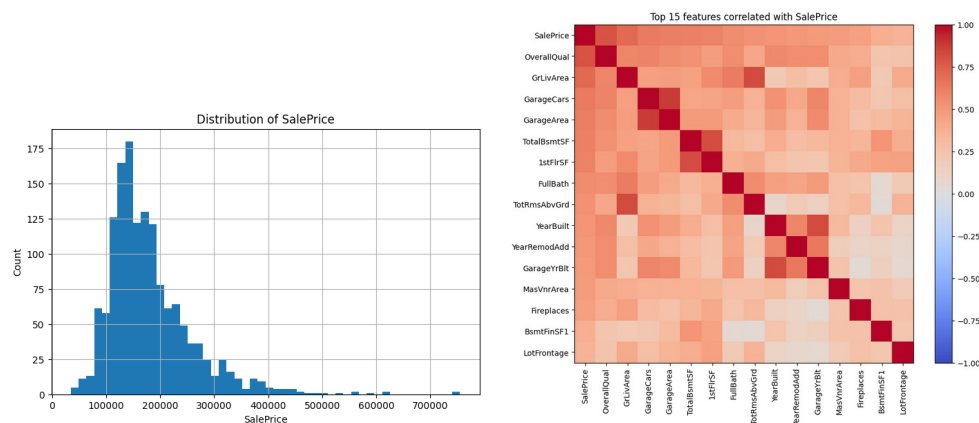
Given structural, quality, and location features of a home, predict its sale price. This is the Kaggle "House Prices — Advanced Regression Techniques" competition dataset: 1,460 homes sold in Ames, Iowa between 2006 and 2010, described by 79 raw features spanning lot size, quality ratings, basement and garage details, and neighborhood (De Cock, 2011).

It is a deliberately good foundational project because it forces the full regression lifecycle without needing anything beyond classical machine learning: a right-skewed target, a mix of numeric, categorical, and ordinal features, missingness that mostly means something rather than being random, and two genuine outliers documented in the source literature.

Kaggle's competition data requires an account and API token to download directly. Rather than gate the whole pipeline on that, the loader prefers a manually downloaded file if one is present, but falls back to fetching the identical dataset through scikit-learn's OpenML mirror — same 1,460 rows, 81 columns, target included. The result: the entire pipeline runs end to end from a clean clone with zero credentials.

2. Data & Exploratory Analysis

The target, SalePrice, is right-skewed: a small number of very expensive homes stretch the distribution's tail. This single fact drives a modeling decision used throughout the project — every model is trained on $\log_{1p}(\text{SalePrice})$ rather than the raw price, so that the handful of expensive-home errors don't dominate the loss function, with predictions inverse-transformed before any dollar-denominated metric is computed.



Left: SalePrice is right-skewed, motivating the $\log_{1p}$ target transform. Right: correlation heatmap of the top features against SalePrice.

Two hypothesis tests, run during EDA, turned out to anchor the rest of the project:

- Overall quality rating (OverallQual) correlates with SalePrice at $r = 0.79$ ($p \approx 0$, essentially certain not to be noise).
- Homes with central air conditioning average \$186,000 versus \$105,000 for homes without (Welch's $t = 17.3$, $p \approx 2 \times 10^{-37}$) — two clearly different price populations.
- The 95% confidence interval for mean SalePrice is \$176,843–\$185,000.

Both findings reappear later: OverallQual turns out to be the single most important SHAP feature, and central-air/quality-adjacent fields dominate the top ten. That consistency between an early statistical test and a late model-explainability result is a useful sanity check that the model learned real signal rather than spurious correlation.

The EDA also surfaced two documented outliers: homes with more than 4,000 square feet of living area (GrLivArea) that sold for under \$300,000 — flagged in the original De Cock paper as data-entry anomalies, not genuine market outcomes. Every other large home in the dataset sold in the \$700,000+ range, consistent with its size. These two rows are dropped before training.

3. Feature Engineering

Three decisions here matter more than the mechanical encoding steps around them:

- Missingness is mostly meaningful, not random. In the Ames data dictionary, a blank value in columns like PoolQC, FireplaceQu, or GarageType means "this house doesn't have that feature," not "value unknown." These are filled with an explicit "None" category (or 0 for the matching numeric columns) before any statistical imputation runs. Only genuinely missing values — LotFrontage, Electrical — fall through to median/mode imputation, with LotFrontage specifically imputed by neighborhood median, since frontage varies by area but is fairly consistent within one.
- Ordinal encoding, not one-hot, for quality scales. Columns like KitchenQual or ExterQual have a real order (Poor < Fair < Typical < Good < Excellent). One-hot encoding would throw that ordering away and force the model to re-learn it from scratch across separate columns; mapping to integers 0–5 preserves it directly and shrinks the feature count. Genuinely nominal categoricals — Neighborhood, SaleCondition — still get one-hot encoded, since there is no natural order to them.
- Split before fitting the pipeline, not after. Raw rows are split into train and test first; the imputer, encoder, and scaler are fit only on the training split and merely applied (never re-fit) to the test split. Fitting on the full dataset before splitting is a common and easy-to-miss leakage bug — it lets test-set statistics quietly leak backward into training.

After outlier removal (1,460 → 1,458 rows) and encoding, the feature set grows from 79 raw columns to 258 numeric features, with zero remaining missing values, ready for modeling.

4. Modeling & Hyperparameter Tuning

Five candidate models were trained and compared with 5-fold cross-validation on the log-transformed target, then evaluated on a held-out 20% test split with metrics converted back to dollars:

Model	CV RMSE (log)	Test MAE	Test RMSE	Test R ²
Lasso	0.114 ± 0.009	\$14,083	\$19,422	0.932
Ridge	0.114 ± 0.010	\$14,489	\$20,183	0.926
XGBoost (tuned)	0.119	\$14,409	\$20,120	0.927
XGBoost (baseline)	0.121 ± 0.010	\$14,773	\$20,236	0.926
Linear Regression	0.127 ± 0.013	\$15,493	\$22,045	0.912
Random Forest	0.140 ± 0.011	\$16,448	\$24,128	0.895

Optuna's 30-trial Bayesian search over XGBoost's hyperparameters improved test MAE from \$14,773 to \$14,409 (a 2.5% reduction) and RMSE from \$20,236 to \$20,120, converging on a shallower, lower-learning-rate configuration (max_depth=4, learning_rate=0.026, n_estimators=544) than the untuned defaults — more, smaller boosting steps generalizing better than fewer, bigger ones.

An uncomfortable but important result: Lasso and Ridge outperformed every tree ensemble on the held-out test set, including tuned XGBoost (R² = 0.932 vs. 0.927). This is a genuine result, not a bug — after the feature engineering above, especially ordinal encoding and the log-target transform, the relationship between features and log(SalePrice) is close to linear, and 1,458 rows curated for a statistics textbook is small and clean enough that a regularized linear model doesn't have much bias left to overcome, while a tree ensemble has more flexibility than the data justifies.

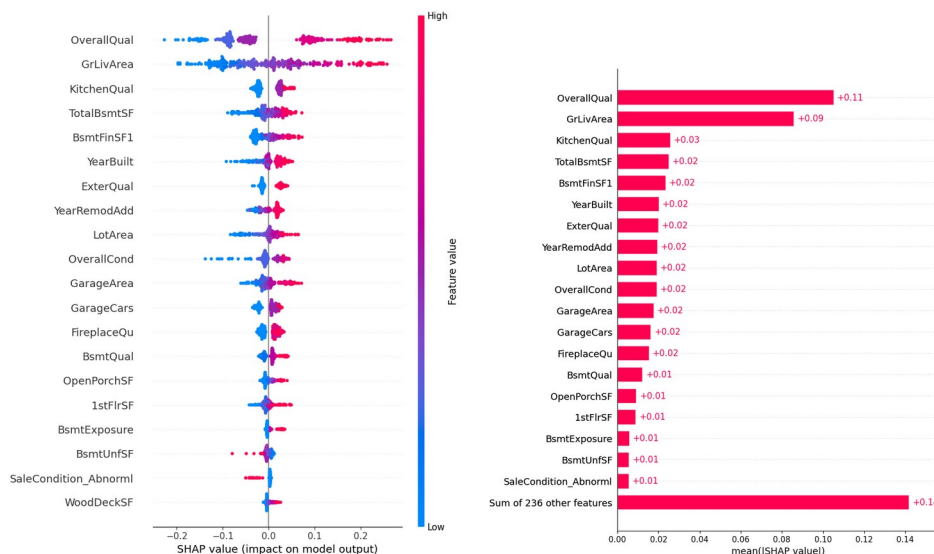
The 5-fold cross-validation standard deviations reinforce the same story: Random Forest had both the highest mean CV error and one of the highest standard deviations — a sign of underfitting a near-linear target with default

hyperparameters rather than overfitting. Lasso and Ridge had the lowest standard deviations, the most stable estimates across folds, mirroring their eventual test-set win.

Yet the served model is XGBoost, not Lasso. That is a deliberate choice for this portfolio project: it exercises the full tuning-and-explainability pipeline (Optuna plus SHAP's TreeExplainer) end to end and demonstrates handling a more complex model family. If this were an actual production decision rather than a demonstration of technique breadth, Lasso would win outright — it is simpler, faster to serve, and scored better on this exact test set. Stating that tradeoff plainly, rather than quietly picking XGBoost and presenting it as the empirically superior model, is itself part of the engineering discipline this project is meant to demonstrate.

5. Explainability

SHAP's TreeExplainer, run on the tuned XGBoost model, confirms the EDA findings from Section 2: OverallQual and GrLivArea dominate feature importance, with KitchenQual and TotalBsmtSF close behind.



SHAP summary plot (left) and mean absolute SHAP value per feature (right), tuned XGBoost model.

This consistency between an independent statistical test run early in the project and a model-internal explainability method run at the end is a genuinely useful check — it means the model's most important features are the same ones a human analyst would flag from the data directly, not artifacts of how the model happens to fit.

6. Serving & Deployment

A FastAPI /predict endpoint accepts a partial JSON dictionary of raw feature names — nobody hand-typing a request wants to fill in PoolQC and MiscFeature. Fields left out are reindexed to missing and pass through the same imputation used during training, so a request with just OverallQual and GrLivArea still returns a reasonable prediction built on sensible defaults for everything else.

The fitted pipeline, the trained model, and the exact list of training columns are persisted together as three artifacts, so inference never silently drifts from the preprocessing path used during training. A Dockerfile bakes a freshly trained model into the image at build time, producing a self-contained container with no external artifact store required — deployable to Railway or Render once pointed at the correct root directory within the monorepo.

An interactive HTML architecture diagram and an animated GIF (both included with the project) walk through this same pipeline stage by stage, showing the real metrics produced at each step.

7. Case Study — An Automated Valuation Model for a Regional Real Estate Brokerage

Consider a hypothetical mid-sized regional brokerage — call it Meridian Realty Group — operating across a metro area with a few hundred active listings at any time. Sellers routinely ask agents for a ballpark price before committing to a listing agreement, and in a competitive market, whichever agent responds first with a credible number often wins the listing.

The proposed system: a homeowner fills in a short web form — lot size, self-assessed condition, year built, neighborhood, square footage — and the FastAPI backend from Section 6 returns an instant estimated price. The listing agent uses that number as an opening conversation point, not a legal appraisal; a licensed, in-person appraisal remains mandatory before closing. The value the model adds is speed: an estimate in seconds instead of a multi-day comparative market analysis (CMA) that traditionally requires an agent to manually pull comparable sales.

How good is good enough? The tuned model's MAE of \$14,083 against an average sale price of roughly \$180,921 works out to about 7.8% relative error. That is comparable to the error bands publicly reported by consumer automated valuation models such as Zillow's Zestimate, which has historically cited a median error in the high single digits for off-market homes. An instant estimate in that range is credible enough to open a conversation, while being explicit with homeowners that it is a starting estimate, not a final number.

Explainability as a trust mechanism: a common complaint about automated valuation models is that they produce a number with no visible reasoning. Because the served model supports SHAP explanations (Section 5), the same infrastructure can surface a plain-language reason alongside the estimate — "this estimate is driven primarily by your home's overall quality rating and living area" — addressing that opacity complaint directly and giving the listing agent something concrete to discuss with the seller.

Where the Lasso-vs-XGBoost tradeoff becomes a real decision: a brokerage optimizing purely for the tightest possible error band would serve Lasso, per the test-set results in Section 4. A brokerage that wants to market "AI-explained pricing" as a differentiator against competitors, and is willing to trade a small amount of accuracy for the richer per-feature explanations that a tree-based SHAP analysis supports well, might deliberately choose to keep XGBoost in production. Neither choice is wrong — it depends on whether the business is selling precision or selling a transparent, defensible number.

Caveat: the Ames dataset reflects a single Iowa metro area's market conditions circa 2006–2010. A real deployment would need geographically and temporally local retraining — home prices in a different metro, or a decade later, follow different relationships between features and price — plus mandatory human review before anything customer-facing is published as an official number.

Part II — Customer Churn Prediction

Imbalanced Classification and Cost-Sensitive Decision-Making on Telecom Data

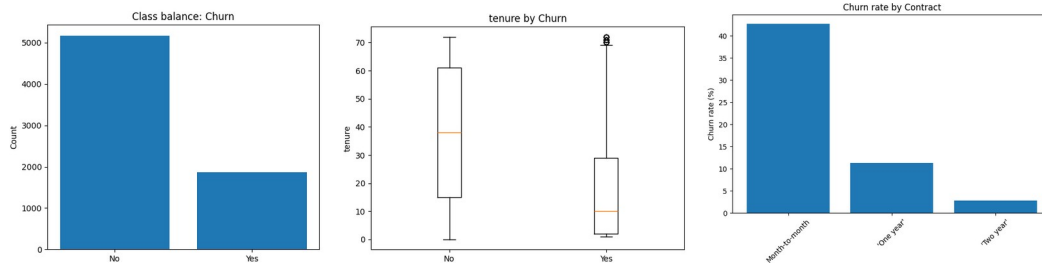
8. Problem Statement & Business Context

Predict which telecom customers are about to cancel their service, using account and service details: tenure, contract type, monthly charges, and which add-ons they've subscribed to. This is the IBM Telco Customer Churn dataset — 7,043 customers, a 26.5% churn rate.

The interesting engineering work in this project isn't fitting a classifier — that part is routine. It's everything imbalanced classification requires beyond calling `.fit()`: handling a minority class deliberately rather than by accident, choosing a decision threshold on purpose instead of defaulting to 0.5, and translating model output into a cost/benefit tradeoff a non-technical stakeholder can actually act on. As with Project 1, the dataset is fetched through a credential-free OpenML mirror when a manually downloaded copy isn't present.

9. Data & Exploratory Analysis

The churn rate is 26.54% (95% Wilson confidence interval: 25.5%–27.6%) — imbalanced, but not extreme. That distinction matters: it means both class-weighting and synthetic oversampling are viable strategies here, where a dataset with a much rarer positive class (say, 0.1% fraud) would rule out plain class weighting much sooner.



Class balance, tenure by churn status, and churn rate by contract type.

- Tenure is the strongest signal by a wide margin: churned customers average 18.0 months tenure versus 37.6 months for retained customers (Welch's $t = 34.8$, $p \approx 1.2 \times 10^{-232}$). New customers churn; customers who've stuck around for years mostly keep sticking around.
- Contract type is hugely predictive ($\chi^2 = 1184.6$, $p \approx 6 \times 10^{-258}$ against Churn). This is exactly why Contract is ordinal-encoded — Month-to-month < One year < Two year — rather than one-hot encoded: the order itself carries information a model can use directly, rather than three independent categories that happen to be correlated.

10. Feature Engineering & Imbalance Handling

One data quirk needed fixing before anything else: eleven customers, all with zero months of tenure, store their TotalCharges field as a literal blank string rather than a number, because they haven't been billed yet. Coercing the column to numeric turns those into missing values, which are then filled with 0 — not the column median — since zero genuinely is the correct total charge for a brand-new customer, not an unknown quantity.

Binary Yes/No fields (Partner, Dependents, PhoneService) are mapped directly to 0/1; Contract gets the ordinal treatment described above; everything else genuinely nominal (InternetService, PaymentMethod, the OnlineSecurity/TechSupport family) is one-hot encoded. The train/test split is stratified, preserving the ~26.5% churn rate in both splits — an unlucky plain random split could otherwise shift the class balance meaningfully between train and test at this sample size.

Two imbalance-handling strategies were benchmarked on the same tuned model architecture:

Strategy	Precision	Recall	F1	ROC-AUC
class_weight='balanced'	0.607	0.666	0.635	0.841
SMOTE oversampling	0.533	0.759	0.626	0.839

The two land at nearly the same ROC-AUC but at very different points on the precision/recall tradeoff. Class weighting penalizes misclassifying the minority class during training without inventing new data points; SMOTE synthesizes new minority-class examples by interpolating between real ones, pushing recall up at precision's expense. Neither is objectively better — the right choice depends on the relative cost of a false positive versus a false negative, which is exactly the question Section 12 answers directly in dollar terms.

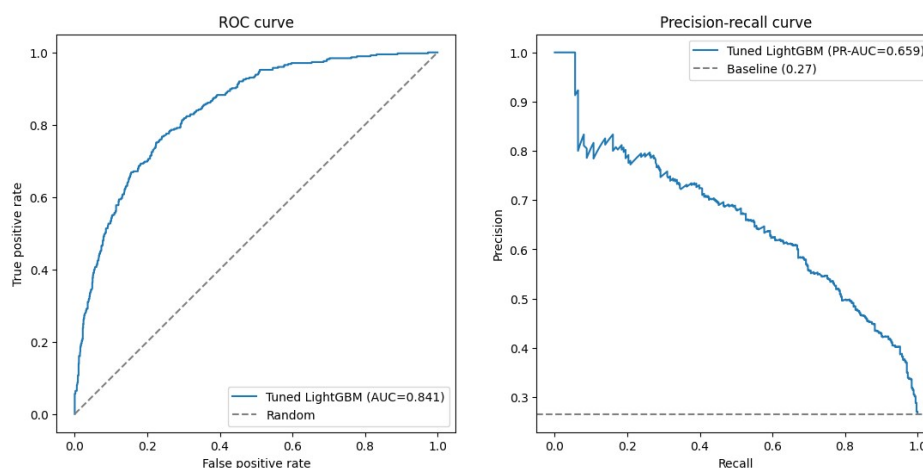
11. Modeling & Hyperparameter Tuning

Four candidate classifiers were compared with stratified 5-fold cross-validation, scored on ROC-AUC, then evaluated on a held-out 20% stratified test split:

Model	CV ROC-AUC	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.840 ± 0.014	0.584	0.671	0.624	0.837
CatBoost	0.837 ± 0.011	0.491	0.813	0.612	0.828
LightGBM (baseline)	0.831 ± 0.011	0.515	0.757	0.613	0.826
Random Forest	0.812 ± 0.011	0.512	0.727	0.601	0.812
LightGBM (tuned)	0.843	0.607	0.666	0.635	0.841

Logistic Regression's untuned CV ROC-AUC (0.840) was already competitive with the fully tuned LightGBM model (0.843) — a useful reminder that on a modest-sized, mostly categorical dataset with strong linear signal (tenure, contract type), a simple regularized linear model is a genuinely strong baseline, not a placeholder to beat on the way to something fancier. CatBoost's untuned recall (0.813) was the best of any single untuned model — if recall were the deployment priority, catching as many churners as possible regardless of precision, CatBoost's defaults would be the better starting point.

RandomizedSearchCV (30 iterations, stratified 5-fold, optimizing ROC-AUC) moved LightGBM's test ROC-AUC from 0.826 to 0.841 and F1 from 0.613 to 0.635. The winning hyperparameters favor many shallow, slow-learning trees (`max_depth=2`, `learning_rate=0.032`, `n_estimators=443`) over fewer, deeper ones — the same bias-toward-simplicity pattern that won when tuning XGBoost in Project 1.



ROC curve and precision-recall curve, tuned LightGBM model.

A learning curve (training vs. validation ROC-AUC as training size grows) shows the two curves converging rather than diverging, meaning this model sits closer to underfitting than overfitting at the current data size — more data would likely help more than additional regularization would.

12. Business Cost-Sensitive Evaluation

The headline result of this project: the F1-optimal decision threshold actually costs more than the naive default of 0.5.

Using illustrative cost assumptions — a \$50 retention offer, \$800 in lifetime value lost per uncaught churning, and a 50% success rate for retention offers that do reach a genuine churning — three thresholds were compared on the tuned model's test-set probabilities:

Threshold	Value	Total cost	TP	FP	FN
Default	0.50	\$210,500	296	298	78
F1-optimal	0.64	\$220,100	249	161	125
Cost-optimal	0.23	\$201,250	356	533	18

F1 treats a false positive and a false negative as equally costly by construction. In this business, they clearly aren't — missing a churning is assumed to cost sixteen times more than an unnecessary retention offer. Optimizing for F1 implicitly assumes a 1:1 cost ratio that doesn't hold, so it lands on a threshold that's too conservative about flagging churning risk. The cost-optimal threshold deliberately accepts far more false positives (533 versus 161) to cut false negatives from 125 to 18, saving \$9,250 — 4.4% — versus the naive default.

Caveat, stated plainly: the \$50, \$800, and 50% figures behind this table are illustrative placeholders, not fitted from real business data. Before this reasoning could inform an actual retention budget, those numbers would need to be replaced with a real average customer lifetime value (segmented by contract type and tenure, since those are clearly not uniform across the base), the real cost of whatever retention incentive is offered, and a measured save rate from a past campaign rather than an assumed coin flip. The value of the exercise is the method — optimize the decision threshold against real dollar cost, not a generic classification metric — not this specific dollar figure.

13. Serving — A Streamlit Dashboard

The served interface is an interactive dashboard rather than a bare API: a form exposing all nineteen raw customer fields, submitted to produce a churn probability, a Churn/No-churn decision made at the persisted cost-optimal threshold (not the sklearn default of 0.5), and a SHAP bar chart explaining which features pushed that specific prediction toward or away from churn. Exposing all nineteen fields directly — rather than a reduced subset — means the exact same feature pipeline used during training runs unmodified at inference time, with no separate "lite" preprocessing path to keep in sync.

Verified live in a browser session: a representative customer profile returned a 48.2% churn probability, correctly flagged as "likely to churn" against the 0.23 decision threshold, with the SHAP chart rendering the specific features driving that call.

14. Case Study — Retention Campaign ROI for a Regional Telecom

Consider a hypothetical regional mobile carrier — call it Meridian Mobile — with roughly 500,000 subscribers, wanting to move from a reactive posture (handling cancellation calls as they come in) to a proactive one (identifying at-risk customers and offering retention incentives before they cancel).

Proposed operating model: the full subscriber base is scored monthly through the trained pipeline from Sections 10–11. Customers whose predicted churn probability clears the cost-optimal threshold of 0.23 are routed into a retention campaign; the retention team lead uses the Streamlit dashboard from Section 13 to spot-check individual flagged customers and see the SHAP-based reasoning before approving an offer, rather than acting on an unexplained score.

Extrapolating the test-set savings: on the roughly 1,409-customer held-out test split, choosing the cost-optimal threshold instead of the naive 0.5 default saved \$9,250 — about \$6.56 per customer scored. Scaling that per-customer figure (illustratively, and only by holding the underlying cost assumptions constant, which is a simplification) across a 500,000-subscriber base suggests on the order of \$3.28 million in annual avoided cost,

arising specifically from calibrating the decision threshold to the real cost asymmetry between a missed churning and a wasted retention offer — not merely from the model existing in the first place. A carrier that trained the same model but left the threshold at the sklearn default would leave a meaningful fraction of that value on the table, and one that optimized the threshold for F1 instead of cost would, per Section 12, do worse than the default.

This figure needs the same caveat as Section 12, restated: it rests entirely on the illustrative \$50/\$800/50% cost assumptions. Before it could anchor a real budget conversation, Meridian Mobile's finance team would need to supply actual average customer lifetime value broken out by contract type and tenure band, the actual cost of whatever retention incentive is offered, and a measured redemption/save rate pulled from a past campaign rather than an assumed 50/50 split. What transfers cleanly from this project to a real deployment is the methodology, not the specific number.

Segment-level targeting: since contract length and tenure are the dominant SHAP drivers (Section 9), the retention team can prioritize the most aggressive, most expensive save offers for month-to-month, low-tenure customers, and reserve lighter-touch offers for one- and two-year contract holders who are statistically far less likely to churn regardless of intervention.

Where class-weighting versus SMOTE becomes an operational choice: if the retention team's outreach capacity is the binding constraint — a fixed number of agent calls per month — the higher-precision class-weighted model (Section 10) wastes fewer of those scarce calls on customers who weren't going to churn anyway. If outreach is cheap and automated (a targeted email campaign) and only the human review step is scarce, SMOTE's higher recall — catching more true churners at the cost of more false alarms in the queue — may be the better fit. The same precision/recall tradeoff surfaced technically in Section 10 resolves differently depending on which resource is actually scarce in the business.

Cross-Project Lessons

Four patterns recur across both projects, despite one being regression and the other imbalanced classification:

- The same architecture shape works for both problem types: credential-free data loading, EDA paired with formal hypothesis tests, a leakage-free feature pipeline persisted as a single artifact, multi-model comparison with cross-validation, automated hyperparameter search, SHAP explainability, and a served interface. None of that shape is regression-specific or classification-specific.
- The fancier model is not automatically the better model. Lasso beat a tuned XGBoost on house prices; untuned Logistic Regression nearly matched a fully tuned LightGBM on churn. On small-to-medium tabular datasets with real linear signal after good feature engineering, a regularized linear model is a genuinely strong baseline in both projects, not a formality to clear before reaching for something more complex.
- Both hyperparameter searches converged on the same shape of answer: shallower trees, lower learning rates, more boosting rounds — more, smaller steps generalizing better than fewer, larger ones. That is a reasonable prior to start from on any small tabular dataset, not a coincidence specific to either project.
- Explainability is also a consistency check, not just a deliverable. In both projects, the features SHAP ranked most important were the same ones flagged by an independent statistical test run during EDA, well before any model existed. That agreement is evidence the models learned real signal rather than an artifact of how each one happens to fit its training data.

Both case studies point at the same underlying truth: the technical metric — R^2 , ROC-AUC, F1 — is necessary but not sufficient. What makes either project defensible to a non-technical stakeholder is translating that metric into a dollar figure or a time saving, stating plainly which assumptions the figure depends on, and being honest about which model would actually be chosen if the goal were the best outcome rather than the broadest demonstration of technique.

Appendix — Technology Stack & Testing

Category	Project 1: House Price	Project 2: Customer Churn
Data access	OpenML fallback (Kaggle-identical)	OpenML fallback (Kaggle-identical)
Feature pipeline	scikit-learn ColumnTransformer	scikit-learn ColumnTransformer
Models compared	Linear, Ridge, Lasso, Random Forest, XGBoost	Logistic Regression, Random Forest, LightGBM, CatBoost
Imbalance handling	N/A (regression)	class_weight='balanced', SMOTE
Hyperparameter search	Optuna (Bayesian, TPE)	RandomizedSearchCV (grid/random)
Explainability	SHAP TreeExplainer	SHAP TreeExplainer
Business layer	N/A	Cost-sensitive threshold optimization
Serving	FastAPI + Docker	Streamlit dashboard
Test suite	36 tests	31 tests

Both projects are part of a larger sixteen-project AI/ML engineering curriculum spanning classical machine learning, deep learning, NLP, computer vision, reinforcement learning, MLOps, and generative AI. Full task checklists, source code, and test suites for both projects live in the `ml-systems` repository.